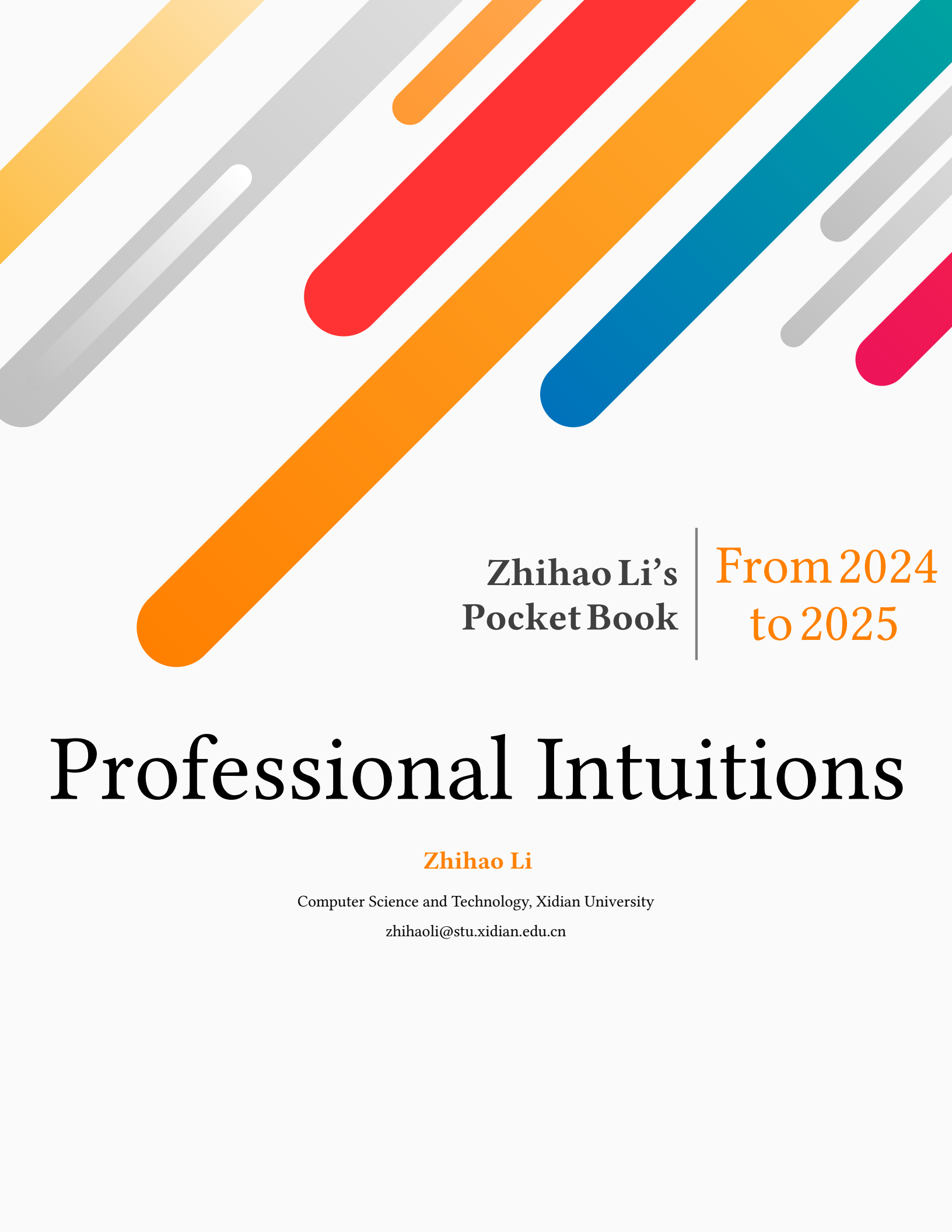




Zhihao Li's
Pocket Book

From 2024
to 2025

Professional Intuitions

Zhihao Li

Computer Science and Technology, Xidian University

zhihaoli@stu.xidian.edu.cn

CONTENTS

CHAPTER 1	PROFESSION & LEARNING	PAGE 4
1.1	Mathematical Principles	4
1.2	Computer Graphics	4
1.2.1	Camera Theory	4
	相机成像	4
	相机内参	5
	视场角 (FoV)	6
	球谐函数 (Spherical Harmonics)	6
	关键帧	7
	UV 坐标系统	7
1.3	Digital Human	8
1.3.1	Model Learning	8
	FLAME	8
CHAPTER 2	PHILOSOPHY & INSIGHT	PAGE 9
2.1	Research Guidelines	9
2.1.1	Research Purpose	9
2.1.2	Global Stages	9
2.1.3	Early Stage	9
2.2	Research Habits	10
2.2.1	Paper Reading	10
	跟进前沿	10
	论文略读	10
	论文精读	10
	论文总结	10
2.2.2	Experiment Exploration	11
2.2.3	Balanced Life	11
2.2.4	Future Planning	11
CHAPTER 3	VIEWS & INSPIRATION	PAGE 12
3.1	Learning With Noisy Labels	12
3.2	3D Face Reconstruction	12
3.3	Talking Head Generation	14
3.3.1	Video Animation	14
3.3.2	Audio Animation	14

CHAPTER 4	CODING & SKILLS	PAGE 16
4.1	Environment Configuration	16
4.1.1	<i>Miniconda</i> Installation On Linux	16
4.1.2	<i>Pytorch3d</i> Installation on Windows	16
4.1.3	<i>Ninja</i> Installation on Linux	17
4.2	Python Debugging	17
4.2.1	<i>Visual Studio Code</i> Debugging Configuration	17
4.3	Bugs with Solution	19
4.3.1	Version Conflict	19
4.4	Coding Tricks	19
4.4.1	Python	19
4.4.2	Cmd On Windows	25

CHAPTER 5	GRAMMAR & WRITINGS	PAGE 26
5.1	Mathematical Symbols in LaTeX	26
5.1.1	集合及向量空间的表示	26
5.1.2	运算符号	26
5.2	Extended Markdown	26
5.3	Elaborating Ideas	27

CHAPTER 6	TEMPLATES & TOOLS	PAGE 28
6.1	Templates	28
6.1.1	Loss Calculation	28
6.1.2	Computer Graphics	29

Chapter 1

Profession & Learning

1.1 Mathematical Principles

1. 仿射变换 (Affine Transformation)

$$price = w_{area} \cdot area + w_{age} \cdot age + b \quad (1.1)$$

仿射变换的特点是通过加权和对特征进行线性变换，并通过偏置项进行平移。

2. 非线性频率压缩

在滤波器设计中将整个模拟频率轴压缩到 π/T 之间，使得 $H_a(s), s = j\Omega$ 压缩为 $\widehat{H}_a(s_1), s_1 = j\Omega_1$ ，可以利用正切变换实现频率压缩模型：

$$\Omega = \frac{2}{T} \tan\left(\frac{1}{2}\Omega_1 T\right) \quad (1.2)$$

这个设计思想实质上利用了正切函数定义域有限、值域无限以及奇函数的性质；推而广之，这种设计可以实现特定的单值压缩方法，也可以实现值域的延展。

一些类似的函数特性，对数函数，指数函数分别适合于定义域、值域取值 $0 \sim 1$ 之间的情况，但是对目标域都有所限制，因此这些函数往往没有正切函数具有优良的特性。

1.2 Computer Graphics

1.2.1 Camera Theory

相机成像

相机成像过程涉及到四个坐标系的变换，世界坐标系经过刚体变换（缩放、旋转、平移）后变成了相机坐标系，再次经过透视投影变成图像坐标系，最后经过仿射变换变成像素坐标系。

$$Z \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \underbrace{\begin{pmatrix} \frac{1}{dX} & -\frac{\cot\theta}{dX} & u_0 \\ 0 & 1 & v_0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}}_{\text{内参矩阵}} \underbrace{\begin{pmatrix} R & T \\ 0 & 1 \end{pmatrix}}_{\text{外参矩阵}} \begin{pmatrix} U \\ V \\ W \\ 1 \end{pmatrix}$$

仿射变换 透视投影 刚体变换

CSDN @思妙想多多财-杰

Figure 1.1: 坐标系变换过程

为了简洁表示，一般都通过内参矩阵和外参矩阵来实现坐标系的转换，内参矩阵就是相机的内参，外参矩阵就是相机的外参。

相机内参

相机内参包括相机的焦距 (Focal Length) 以及主点 (Principal Point)，焦距指的是成像元件的光学中心到成像平面的距离，单位通常是像素；主点是相机图像平面上的主点坐标，也就是图像坐标系的原点，通常是相机的光学中心在图像平面上的投影。主点是相机图像中理想情况下图像中心的位置，通常接近图像的几何中心，但在实际相机中，由于制造偏差，可能不完全重合。

针孔相机模型

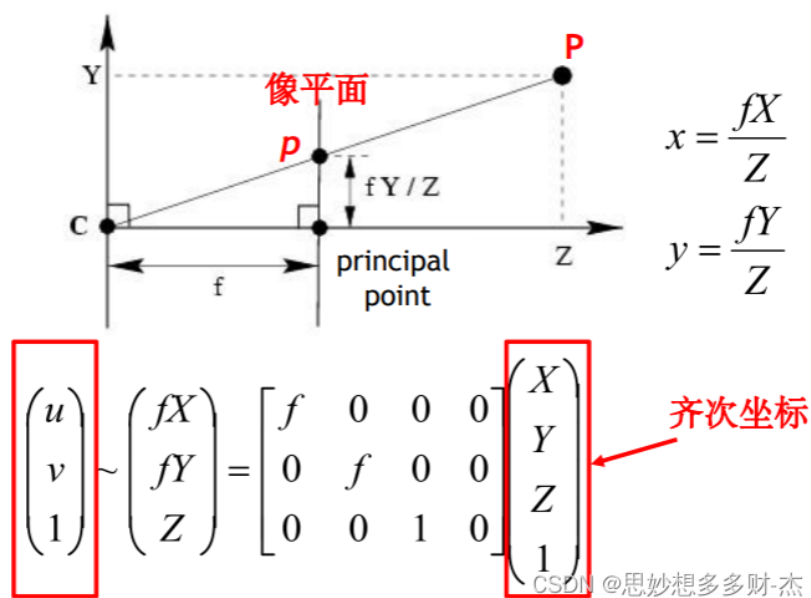


Figure 1.2: 相机内参计算

根据图 1.2 可以计算出对应的内参矩阵，一般来说， f_x 和 f_y 在一个理想的相机中应该是相同的，但是由

于像素的非方形（长宽比不一致）或者相机的几何畸变，这两个值通常是不同的。它们反映了每个像素在水平方向和垂直方向上对应的实际焦距，如公式1.3所示。

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (1.3)$$

- f_x 和 f_y 分别是相机的横向和纵向焦距，在像素单位下的水平和垂直方向上的投影比例系数，表示了相机成像的缩放效果，通常以像素单位进行测量；当像素单元是正方形时， $f_x = f_y$ ，是长方形时， $f_x \neq f_y$ 。
- c_x 和 c_y 是相机光心在像素坐标系下的水平和垂直方向上的位置，表示了相机成像中心在图像上的位置。

视场角 (FoV)

视场角是以光学仪器的镜头为顶点，以被测目标的物像可通过镜头看到的最大范围的两条边缘构成的夹角，称为视场角 (Field of View, FoV)。FoV 指的是相机视场张角，用来描述拍摄的范围。

最常用的是水平视场角 (VFOV) 和垂直视场角 (HFOV)，其与焦距的转换关系如下公式所示。

$$\begin{cases} \tan \frac{VFOV}{2} = \frac{w/2}{f_x} \Rightarrow f_x = \frac{w}{2 \tan \frac{VFOV}{2}} \\ \tan \frac{HFOV}{2} = \frac{h/2}{f_y} \Rightarrow f_y = \frac{h}{2 \tan \frac{HFOV}{2}} \end{cases} \quad (1.4)$$

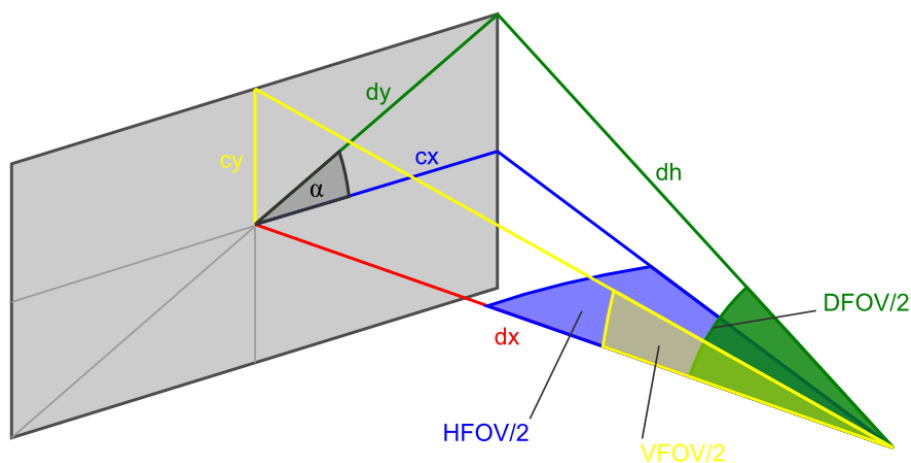


Figure 1.3: FoV 视角分布

球谐函数 (Spherical Harmonics)

球谐函数 (SH) 是一种用于近似球面函数的数学工具，在计算机图形学中广泛用于光照和渲染。

1. 基本概念

球谐函数的阶数 (Degree)

- 0 阶：常数项，表示均匀光照；

- 1 阶：线性变化，可以表示主要方向的光照；
- 2 阶：二次变化，可以表示更复杂的光照；
- 3 阶：更高次变化，可以表示更细致的光照细节。

2. 系数数量

对于 degree 为 L 的球谐函数，总系数数量为 $(L+1)^2$ ，例如当 $L=3$ 时：总系数 = 16 个，包含 0 阶：1 个系数，1 阶：3 个系数，2 阶：5 个系数，3 阶：7 个系数。

3. 光照表示

- 用于表示每个高斯点的方向性光照响应；
- 较高的 degree 可以表示更精细的光照变化；
- 默认值 3 提供了良好的视觉质量和计算效率的平衡。

4. 性能影响

- degree 越高，需要存储的系数越多；
- 计算复杂度随 degree 增加而增加；
- 内存占用也随 degree 增加而增加。

5. 质量与效率权衡

- 较低 degree(0-1)：基础光照，效果简单；
- 中等 degree(2-3)：较好的视觉效果，适中的计算量；
- 高 degree(4+)：更精细的光照细节，但计算成本高。

关键帧

关键帧（代表帧，Key frame）是视频中一个镜头的关键图像帧，可以反映一个镜头的主要内容。关键帧间隔指的是两个关键帧之间间隔的中间帧数量，+ 较小的间隔 = 更细腻动画 + 较大的间隔 = 更粗糙但效率更高

UV 坐标系统

UV 坐标是 3D 模型的二维纹理坐标系统，用于定义如何将 2D 纹理图像映射到 3D 模型表面。

1. 基本概念

- U: 横向坐标 (类似 X 轴)；
- V: 纵向坐标 (类似 Y 轴)；
- 范围：通常在 $[0,1]$ 或 $[-1,1]$ 之间。

2. UV 映射原理

```
1 3D 模型表面          2D 纹理图像
2   ↓                  ↓
3 (x,y,z)  ----映射----> (u,v)
```

3. UV 坐标系统特点

- 二维性: 将 3D 表面”展开”到 2D 平面;
- 连续性: 保证纹理在模型表面连续;
- 无缝性: 避免纹理接缝和扭曲.

1.3 Digital Human

1.3.1 Model Learning

FLAME

```
1 flame_param = {
2     'shape': tensor,          # 形状参数
3     'expr': tensor,           # 表情参数
4     'rotation': tensor,       # 旋转参数
5     'neck_pose': tensor,      # 颈部姿态
6     'jaw_pose': tensor,       # 下巴姿态
7     'eyes_pose': tensor,      # 眼睛姿态
8     'translation': tensor,    # 平移参数
9     'static_offset': tensor,  # 静态偏移
10    'dynamic_offset': tensor  # 动态偏移
11 }
```


Chapter 2

Philosophy & Insight

2.1 Research Guidelines

2.1.1 Research Purpose

“大道至简”，追求 *simple and effective* 研究。学习姚期智院士的选题三问，

1. 你的研究选题真的很重要吗？
2. 有多少人在做？
3. 为什么轮到了你来做？

关于学术品味，姚期智院士曾讲，“一个好的问题应该是漂亮的，即简洁、新颖、令人惊讶；也应该是重要的，即人们关心其答案；并且应该具有普遍吸引力，能够超越学术界限，引起广泛关注”。

- Attractive (漂亮的): Simplicity; Novelty; Surprise;
- Importance (重要的): People care what the answer is;
- Universe (普遍的): Transcends academic boundaries.

2.1.2 Global Stages

1. 不要被工程化的思维所束缚，做 A+B 的工作，着重分析本质，从源头创新。

2.1.3 Early Stage

1. 前期注重多看研究论文，精度并总结，打牢基础，做研究综述。
2. **研究综述**：有多少途径能够实现研究目的，各自是从哪些地方出发的，本质的区别是什么。

2.2 Research Habits

2.2.1 Paper Reading

跟进前沿

关注领域内发展，要跟进最新研究成果，及时搜集最新研究论文。

- 统一的渠道：X-MOL 学术平台并保持邮件推送。
- 分散的渠道：各个期刊会议的官网。

论文略读

1. 阅读题目和整体流程图；
 2. 阅读摘要和 Introduction；
 3. 选择性阅读 Method 模块。
- 阅读原则：明确研究目标、研究问题、解决方案以及方法流程。
 - 总结原则：一段话总结 + 贡献点 + 启发思路。

论文精读

1. 先阅读题目、摘要、Introduction 和 Method；
 2. 再阅读 Related Work；
 3. 选择性阅读实验部分。
- 阅读原则：明确研究目标、研究问题、解决方案以及方法流程。
 - 总结原则：一段话总结 + 手推思路。

论文总结

1. 论文阅读后及时总结，多发掘创新点、启发点；
 2. 积累论文写作经验，可以在阅读论文中总结；
 3. 集中在一个地方整理笔记，形成研究发展体系。
- 略读选择的平台：TexStudio 汇总笔记。
 - 精读选择的平台：TexStudio 汇总笔记 + 纸质笔记。

2.2.2 Experiment Exploration

1. 积累代码实现，技巧、工具库以及模板等；
 2. 多进行代码阅读，关注核心代码的编写；
 3. 关注重要的实验细节，积累调参技巧。
- 积累选择的平台：TexStudio 汇总笔记。

2.2.3 Balanced Life

1. 调控健康的生活节奏，尽量不要熬夜、不要懒觉；
2. 安排适量的运动，保证每周都能有三到五次的运动锻炼；
3. 注重健康饮食，忌高油高盐，保持均衡的营养摄入。

2.2.4 Future Planning

1. 提起规划未来半年到一年的安排；
2. 半年内的计划需清晰具象；
3. 一年内的计划可模糊笼统。

Chapter 3

Views & Inspiration

3.1 Learning With Noisy Labels

1. Co-teaching: Robust Training of Deep Neural Networks with Extremely Noisy Labels

- Views: DNN 的相互指导学习机制，两个模型分别动态地选取一些干净样本相互提供给对方进行学习，目的是过滤不同类型的噪声；
- Inspiration: 直觉是同辈相互纠错的学习机制，而且训练中其样本选取的动态性值得一提；

2. DivideMix: Learning with Noisy Labels as Semi-supervised Learning

- Views: 对噪声数据集进行划分，并同时学习两个模型进行相互指导、标签集成，以克服不同类型的噪声；
- Inspiration: Mutual Learning 和交互学习一定程度上可以增强模型鲁棒性；

3.2 3D Face Reconstruction

1. GaussianAvatars: Photorealistic Head Avatars with Rigged 3D Gaussians

- Views:
GaussianAvatars 一种新颖的方法能够生成表情、姿势和视角完全可控的逼真的头像，核心的思想是将基于 3DGS 的动态 3D 表示与参数化可形变人脸模型进行绑定。这种结合实现逼真渲染的同时能够允许高精度肖像控制，通过内在的参数模型，例如来自驱动序列的表情迁移以及调节形变模型的参数。
- Contributions:
 - 提出了 GaussianAvatars，一种新的结合 3DGS 与参数化人脸模型的人头肖像生成方法。
 - 设计了一个约束性继承策略，能够支持添加和移除 3D 高斯以实现自适应控制。
- Reproduciton:
 - 原始数据：
一个 subject 在一种表情/姿态下，对应 16 个相机 views 拍摄的视频序列，每一个视频序列均 73 FPS 持续 5s，包含 365 帧图像，分辨率为 2200× 3208.

– 数据处理：

- * 从视频序列中提取帧数据，施加下采样的缩放比例 2,4 并移除图像的背景。
- * 针对下采样比例为 4 的图像数据，利用 LandmarkDetectorSTAR 进行关键点检测。然后再所有帧数据上随机采样进行序列跟踪，优化每一帧人脸的姿势和表情参数。初始化参数在第一帧图像上，然后对第一帧图像的刚性形变、全局的关键点以及 RGB 纹理和全局参数以及静态位移都进行 500 步的优化迭代。然后在剩余的全部帧图像上进行序列跟踪。最后通过随机选择一帧图像就行全局跟踪，优化所有的参数。
- * 以 NeRF 格式导出序列跟踪的结果。

– 模型准备：

首先初始化 Gaussian 模型的各个高斯球属性，球谐函数阶数、位置、不透明度、缩放以及旋转等参数。然后根据形状和表情参数初始化 FLAME 模型，包括牙齿参数。最后初始化绑定参数，包括绑定关系以及三角形面元中心位置、缩放以及朝向。

– 场景准备：

首先加载每帧图像的信息，以及相机参数，然后加载预先获取的 FLAME 参数，统一构造出场景信息。然后高斯模型加载场景的 Mesh 网格信息，然后从 PCD 中创建高斯点云。

– 模型训练：

初始化优化参数（FLAME 和 Gaussian 属性）后，从数据集中随机选择一帧图像，根据时间步选择对应帧的 FLAME 参数，通过 FLAME 模型 forward 得到顶点属性，并更新现有的网格属性。然后通过高斯微分光栅化器进行场景渲染，得到渲染的图像和可见性过滤器等属性。接着计算各个损失项，并判断是否需要致密和裁剪操作。最后计算梯度，进行参数的更新。

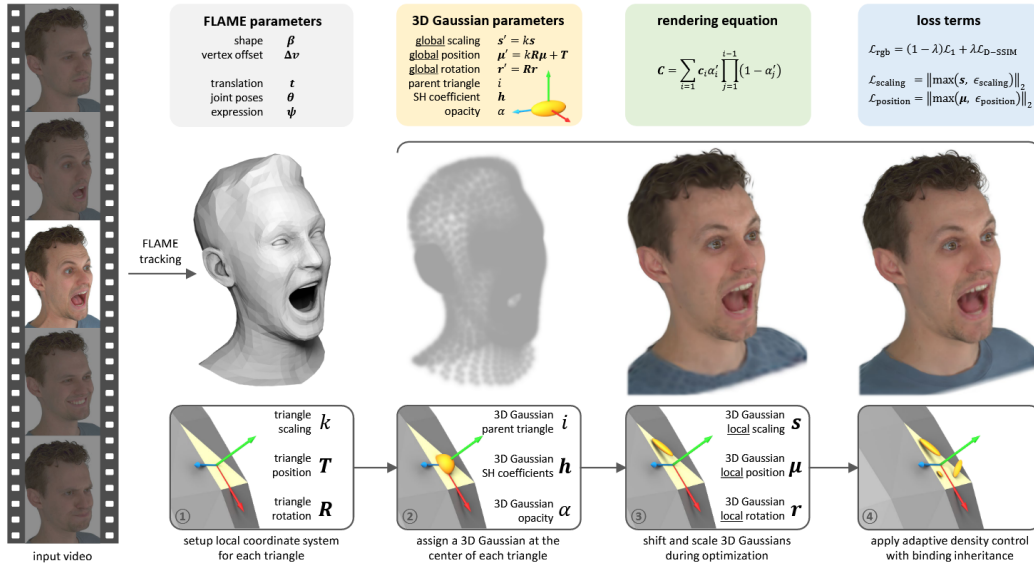


Figure 3.1: GaussianAvatars

3.3 Talking Head Generation

3.3.1 Video Animation

Video-driven 的方法中肖像的面部运动是基于另外一个驱动视频生成的，保证运动的同步性是一个关键点。

1. LivePortrait: Efficient Portrait Animation with Stitching and Retargeting Control

- Views: 作者不同于主流基于扩散模型的方法，提出了基于隐式关键点的视频驱动肖像动画框架，能够明显地增强生成质量和泛化性能，并利用小型 MLP 设计了一个先进的缝接模块和两个重定向模块实现平衡计算效率和可控性的目的。
- Roadmap: 基于 Face vid2vid，核心模块是构建的形变场。
- Inspiration: 采用隐式关键点以及小型 MLP 给模型计算效率带来新的突破。

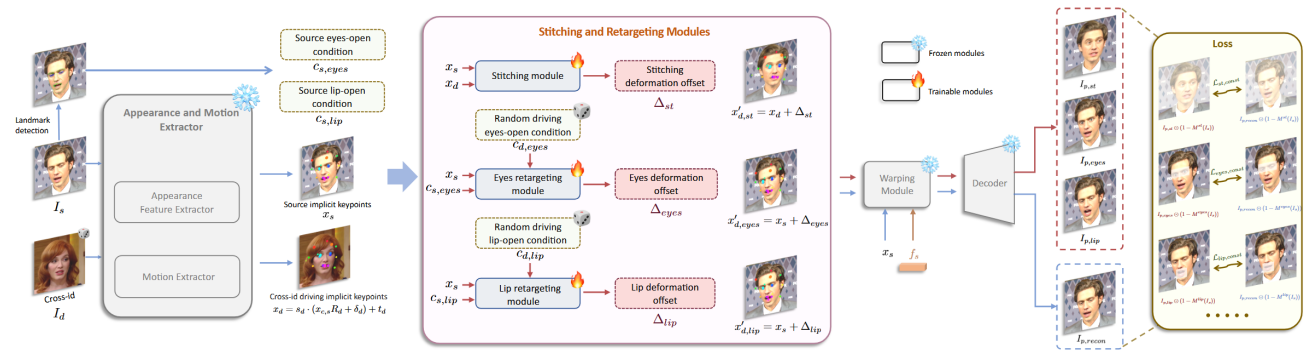


Figure 3. Pipeline of the second stage: stitching and retargeting modules training. After training the base model in the first stage, we freeze the appearance and motion extractor, warping module and decoder. Only the stitching module and the retargeting modules are optimized in the second stage. Please refer to Sec. 3.3 for details.

Figure 3.2: LivePortrait

3.3.2 Audio Animation

Audio-driven 的方法中，肖像的面部运动是直接由音频输入预测的，确保真实和自然性是一个关键点。

1. TalkingGaussian: Structure-Persistent 3D Talking Head Synthesis via Gaussian Splatting

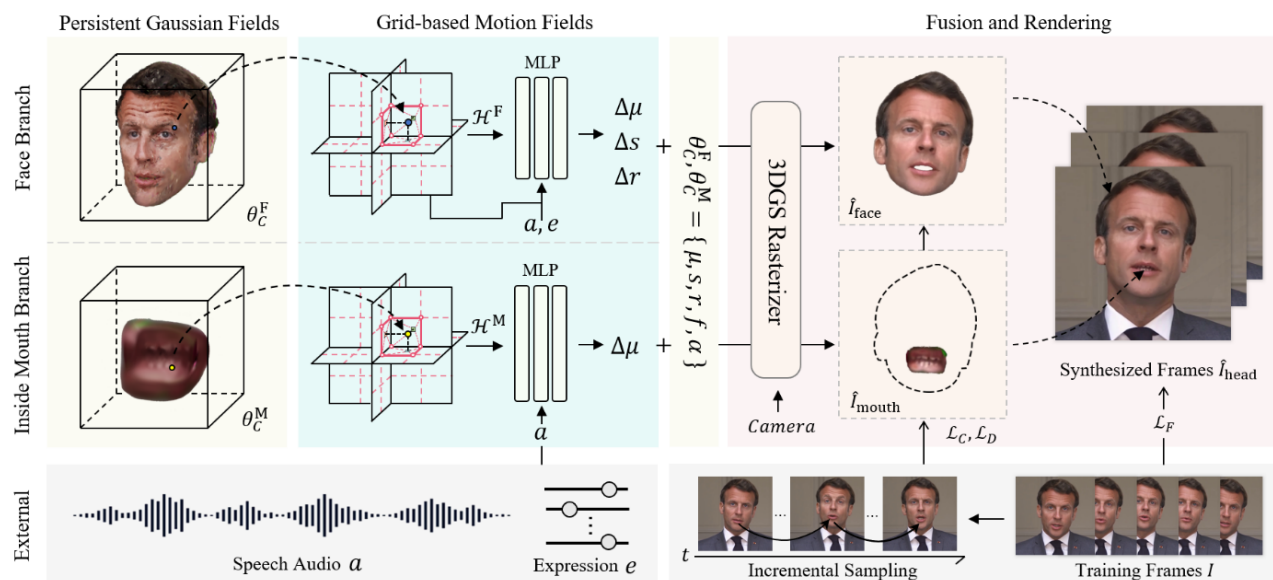


Fig. 2: Overview of TalkingGaussian. Learning from the speech video with training frames I , TalkingGaussian builds two separate branches to represent the dynamic face and inside mouth areas. Queried by the primitives in Persistent Gaussian Fields with parameters θ_C , a point-wise deformation can be predicted from Grid-based Motion Fields conditioned with audio feature a and upper-face expression e . After that, the 3DGS rasterizer renders the deformed 3D Gaussian primitives into 2D images observed from the given camera, which are then fused to synthesize the entire talking head.

Figure 3.3: TalkingGaussian

- 研究目标: talking head synthesis
- 研究问题: 解决对迅速变化外观的不准确预测带来的固有的面部扭曲问题。
- 解决方案: 作者采用基于形变的网络, 对姿势一致性的人头结构施加预测的形变位移, 而不是直接预测形变后的人头, 并提出了一个 Face-Mouth 分解模块, 采用两个相互独立的网络分别学习人脸和嘴唇的形变。
- 启发思路: 施加预测的形变位移, 模块的分解能够提供更加准确的控制而且易于学习。
- 技术路线: 基于 3DGS 构造高斯球面, 首先学习一个基本的粗略平均场, 然后通过 MLP 预测对应的位移以获得形变后的形变场, 优化每个高斯球的属性实现最佳效果, 再利用 3DGS 的光栅化器, 计算逐像素的颜色, 最终渲染出图像。

Chapter 4

Coding & Skills

4.1 Environment Configuration

4.1.1 Miniconda Installation On Linux

```
1  # Fetch the miniconda script
2  export HOME=$PWD
3  wget -q https://repo.anaconda.com/miniconda/ \
4  Miniconda3-py37_4.12.0-Linux-x86_64.sh -O miniconda.sh
5  sh miniconda.sh -b -p $HOME/miniconda3
6  rm miniconda.sh
7  export PATH=$HOME/miniconda3/bin:$PATH
8
9  # Initialize conda
10 source $HOME/miniconda3/etc/profile.d/conda.sh
11 hash -r
12 conda config --set always_yes yes --set changeps1 yes
13
14 # Create new environment
15 conda create -n my_env python=3.8
16 conda activate my_env
```

4.1.2 Pytorch3d Installation on Windows

Windows 下 3D Vision 环境配置 (GaussianAvatars)


```

18 "pythonPath": "D:/anaconda3/envs/d2l/python.exe",
19 "env": {}, // 环境变量字典，可以在这里添加自定义环境变量
20 // 如果需要从文件加载环境变量，可以指定.env 文件的路径
21 "envFile": "${workspaceFolder}/.env",
22 // 是否在程序启动时立即暂停，以便在第一行代码之前设置断点
23 "stopOnEntry": false,
24 "showReturnValue": true, // 是否在调试过程中显示函数的返回值
25 "redirectOutput": true // 是否将程序输出重定向到调试控制台，而不是终端
26 }
27 }
28 }

```

在完成以上准备工作后，设置程序断点，然后直接点击调试三角形按钮，即可开始调试，如图 4.2 所示。

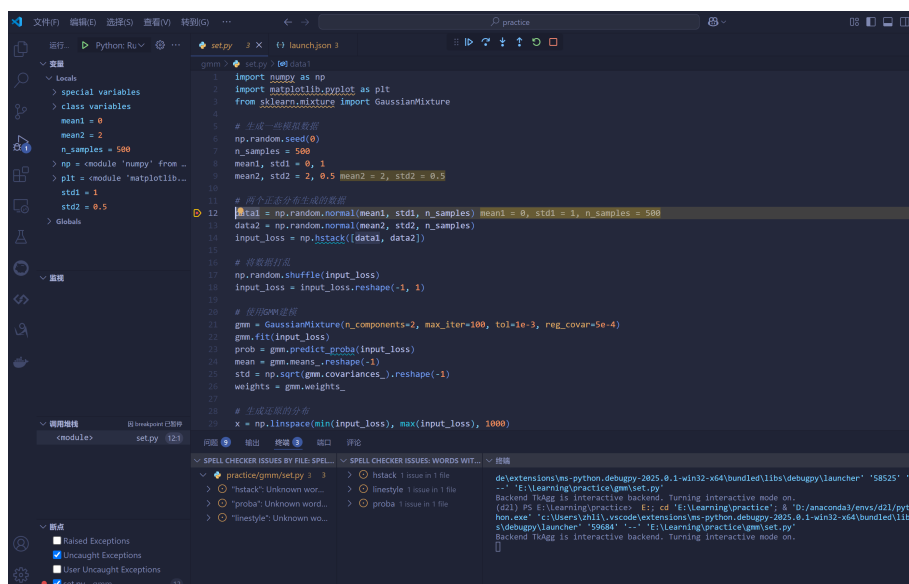


Figure 4.2: Python 程序调试示例

【内嵌调试终端找不到环境变量解决办法】

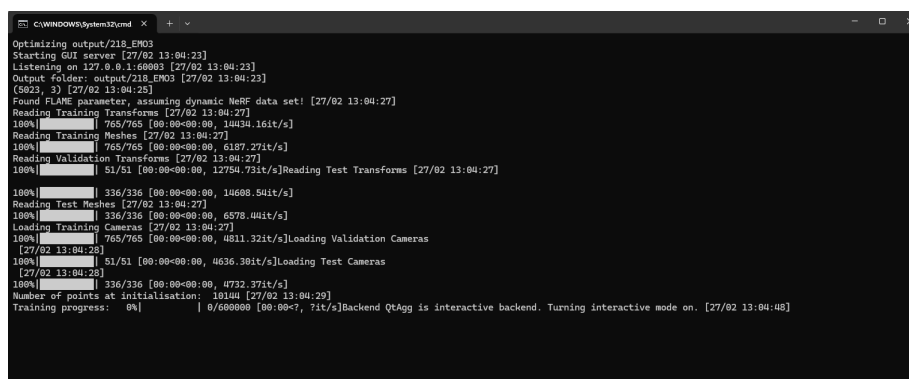


Figure 4.3: 弹出的外部终端

在一些情况下, VS Code 中调试程序无法找到环境变量, 但在 CMD 终端中可以成功运行。这样的情况, 解决办法也很简单, 将调试的终端更改为外部终端即可, 即配置 "console": "externalTerminal" 参数。配置完成后, 重新调试程序, 会调用外部的终端作为调试终端, 如图4.3所示。

4.3 Bugs with Solution

4.3.1 Version Conflict

1.
 - Q: Numpy 版本和 Pytorch 版本不兼容。
 - A: 尝试更换 Numpy 版本。

```
1  assert (self.faces==torch.from_numpy(flame_model.f.astype( \
2  'int64'))).all()
3  RuntimeError: Numpy is not available
```

2.
 - Q: PIL 版本更新问题。
 - A: 在 pillow 的 10.0.0 版本中, ANTIALIAS 方法被删除了, 通过命令 `pip install Pillow==9.5.0` 安装旧版本即可。

```
1  AttributeError: module 'PIL.Image' has no attribute 'ANTIALIAS'
```

4.4 Coding Tricks

4.4.1 Python

1. 路径管理库 pathlib

- 类型指定: `input: Path`
- 路径赋值: `'../../dir'`
- 判断路径是否存在: `Path.exists()`
- 获取父目录: `Path.parent`
- 通配符匹配: `Path.glob()`

2. 数字可读性 8_000: Python 中数字表示含有下划线时, 仅用于提高大数字的可读性, 对数值没有影响。

3. 解析指定命令行参数 parser

```
1  args = parser.parse_args(sys.argv[1:])
```

其中, `sys.argv[0]` 是 Python 脚本名称, `sys.argv[1]` 包含所有实际的命令行参数。例如, 启动命令为, `python train.py --batch-size 32 --learning-rate 0.001` 对应, `sys.argv[0]` 表示 `train.py`, `sys.argv[1]` 表示 `['--batch-size', '32', '--learning-rate', '0.0']`。

4. with 同时打开多个文件

```

1      with open(scene_info.ply_path, 'rb') as src_file, \
2          open(os.path.join(self.model_path, "input.ply"), 'wb') \
3          as dest_file:
4          dest_file.write(src_file.read())

```

5. tensor.scatter_add_ 函数用法

scatter_add_ 是 PyTorch 中的一个张量操作函数，用于将一个张量中的值按照指定的索引累加到另一个张量中。

```

1      torch.Tensor.scatter_add_(dim, index, src) -> Tensor

```

scatter_add_ 会根据 index 中的索引，将 src 中的值累加到目标张量（即调用该方法的张量）的指定位置。

下面代码实现了绑定计数的功能，根据绑定关系（下标索引）将全 1 向量统计到每个面中。

```

1      # 创建计数器和绑定关系
2      counter = torch.zeros(5, device="cuda") # 假设有 5 个面
3      # 4 个点绑定到不同的面
4      binding = torch.tensor([0, 2, 2, 1], device="cuda")
5      # 使用 scatter_add_ 统计每个面绑定的点数
6      counter.scatter_add_(0, binding,
7                          torch.ones_like(binding, dtype=counter.dtype, device="cuda"))
8
9      print(counter) # 输出: tensor([1, 1, 2, 0, 0])

```

6. register_buffer 作用

register_buffer 是 PyTorch 中 nn.Module 的一个方法，用于注册不需要计算梯度的张量。

```

1      def register_buffer(self, name: str, tensor: Optional[Tensor], \
2                          persistent: bool = True)

```

register_buffer 的这些数据是模型状态的一部分，但不需要梯度；在模型移动到 GPU 时会自动转移，会被保存在模型的 state_dict 中。

通过注册缓冲变量后，可以当作属性直接访问使用：

```

1      self.register_buffer(
2          "v_template",
3          to_tensor(to_np(flame_model.v_template), dtype=self.dtype)
4      )

```

```

5 # 1. 直接访问
6 vertices = self.v_template
7
8 # 2. 获取所有 buffers
9 for name, buffer in self.named_buffers():
10     print(f"Buffer {name}: {buffer.shape}")

```

最后总结一下，torch.nn 库中 Parameter 和 Buffer 的区别：

- Parameter: 需要梯度计算，会被优化器更新，用于模型权重。
- Buffer: 不需要梯度，不会被优化器更新，用于常量或中间状态。

7. @dataclass 装饰器使用

@dataclass 是 Python 3.7 引入的一个装饰器，用于自动生成数据类的特殊方法，比如 `__init__`、`__repr__`、`__eq__` 等，常用作配置类、数据传输对象以及不可变数据结构。

```

1 from dataclasses import dataclass
2
3 # 不使用 @dataclass
4 class PersonWithoutDecorator:
5     def __init__(self, name: str, age: int):
6         self.name = name
7         self.age = age
8
9     def __repr__(self):
10         """
11         用于生成字符串表示，主要控制输出格式。
12         """
13         return f"Person(name='{self.name}', age={self.age})"
14
15     def __eq__(self, other):
16         """
17         实现相等比较，重载 == 运算符。
18         """
19         return self.name == other.name and self.age == other.age
20
21 # 使用 @dataclass - 自动生成上述所有方法
22 @dataclass
23 class Person:
24     name: str
25     age: int
26

```

```

27 @dataclass(frozen=True) # frozen=True 使对象不可变
28 class Point3D:
29     x: float
30     y: float
31     z: float
32
33 PWD = PersonWithoutDecorator("Zhihao Li", 22)
34 PWDold = PersonWithoutDecorator("Zhihao Li", 23)
35 Per = Person("Zhihao Li", 22)
36 point = Point3D(1, 2, 3)
37
38 # 判断比较以及装饰器
39 print(PWD)
40 print(PWD == PWDold)
41 print(Per)
42
43 # 判断不可变数据结构
44 print(point)
45 try:
46     point.x = 3
47     print("successfully modified the data")
48     print(point)
49 except:
50     print("not support modifying the data")
51     print(point)
52
53 # results
54 Person(name='Zhihao Li', age=22)
55 False
56 Person(name='Zhihao Li', age=22)
57 Point3D(x=1, y=2, z=3)
58 not support modifying the data
59 Point3D(x=1, y=2, z=3)

```

8. field 函数用法

```

1 class Config(Mini3DViewerConfig):
2     pipeline: PipelineConfig = field(default_factory=PipelineConfig)

```

从上面可以分析得出, PipelineConfig 是一个类, 每个 Config 实例都需要一个独立的 PipelineConfig 实例, 如果直接写 pipeline: PipelineConfig = PipelineConfig(), 所有 Config 实例会共享同一个 PipelineConfig

实例。

因此 field 函数主要用于在初始化中生成动态的实例或数据，常见于类初始化中。

```
1 from dataclasses import field
2
3 field(
4     default_factory=PipelineConfig,  # 用于生成默认值的工厂函数
5     init=True,                       # 是否作为 __init__ 参数
6     repr=True,                      # 是否包含在 repr 中
7     compare=True,                   # 是否参与比较
8     hash=None,                     # 是否参与哈希计算
9     metadata=None                   # 字段元数据
10 )
```

```
1 from dataclasses import dataclass, field
2
3 # 可变默认值
4 @dataclass
5 class Example:
6     list_field: list = field(default_factory=list)
7     dict_field: dict = field(default_factory=dict)
8
9 exp1 = Example([1,2], {"1":3})
10 exp2 = Example([2,2], {"1":3, "2":3})
11 print(exp1)
12 print(exp2)
13
14 exp1.list_field.append(3)
15 print(exp1)
16
17 # 定义对象实例
18 class Number:
19     def __init__(self):
20         self.number = 0
21
22 @dataclass
23 class Config:
24     number: Number = field(default_factory=Number)
25
26 cfg1 = Config()
```

```

27 cfg2 = Config()
28 cfg1.number.number = 1
29 cfg2.number.number = 2
30 print(cfg1.number.number)
31 print(cfg2.number.number)
32
33 # results
34 Example(list_field=[1, 2], dict_field={'1': 3})
35 Example(list_field=[2, 2], dict_field={'1': 3, '2': 3})
36 Example(list_field=[1, 2, 3], dict_field={'1': 3})
37 1
38 2

```

9. tyro.cli 函数用法

tyro.cli() 是一个用于将 dataclass 转换为命令行参数的工具。

预先定义的 dataclass 后，通过语句 `cfg = tyro.cli(Config)` 将其转换为支持命令行参数的配置，之后便可以在命令后中指定对应的参数，作用等同于 `argparse`。

```

1 @dataclass
2 class Config(Mini3DViewerConfig):
3     pipeline: PipelineConfig = field(default_factory=PipelineConfig)
4     cam_convention: Literal["opengl", "opencv"] = "opencv"
5     point_path: Optional[Path] = None
6
7 python local_viewer.py \
8     --point_path path/to/point_cloud.ply \
9     --cam_convention opencv \
10    --sh_degree 3 \
11    --fps 25

```

10. setattr 函数用法

setattr 主要用于设置对象的实例属性。

```

1 class Struct(object):
2     def __init__(self, **kwargs):
3         for key, val in kwargs.items():
4             setattr(self, key, val)
5
6 # 创建实例并设置属性
7 config = Struct(name="FLAME", version="1.0", features=["shape", \

```



```
8         "expression"]])
9
10 # 访问属性
11 print(config.name)      # 输出: "FLAME"
12 print(config.version)   # 输出: "1.0"
13 print(config.features)  # 输出: ["shape", "expression"]
14
15 # 单独使用该方法设置对象 config 的属性
16 setattr(config, "test", "val")
17 print(config.test)
18
19 # results
20 FLAME
21 1.0
22 ['shape', 'expression']
23 val
```

4.4.2 Cmd On Windows

1. 打印当前目录路径 pwd

```
1      PS D:\ZHaoLi\02.Hobbies\01.Blog> pwd
2
3      Path
4      ----
5      D:\ZHaoLi\02.Hobbies\01.Blog
```

Chapter 5

Grammar & Writings

5.1 Mathematical Symbols in LaTeX

5.1.1 集合及向量空间的表示

Rendered Symbol	Source Code	Rendered Symbol	Source Code
S	<code>$\mathcal{S}$</code>	$\mathbb{R}$	<code>$\mathbb{R}$</code>

Table 5.1: LaTeX Symbols and Their Source Code

5.1.2 运算符号

Rendered Symbol	Source Code	Rendered Symbol	Source Code
$\ll$	<code>$\ll$</code>	$\mathbb{R}$	<code>$\mathbb{R}$</code>

Table 5.2: LaTeX Symbols and Their Source Code

5.2 Extended Markdown

在 Markdown 文件中实现下拉式列表，可以通过 ‘html’ 语言实现，具体语法如下：

```
1 <details>
2   <summary> Dependencies (click to expand) </summary>
3
4   ## Dependencies
5   - PyTorch 1.4
6   - matplotlib
7   - numpy
8   - imageio
```

- imageio-ffmpeg
- configargparse

The LLFF data loader requires ImageMagick.
</details>

5.3 Elaborating Ideas

1. 首先阐述问题的必要性，然后引出自己的方法。

Chapter 6

Templates & Tools

6.1 Templates

6.1.1 Loss Calculation

1. L1 Loss

L1 损失也称为平均绝对误差 (MAE)，计算公式如6.1。对异常值不敏感，适合希望减少大误差影响的场景；在零点不可导，优化时需使用次梯度（如符号函数）。

$$L1_{loss} = \frac{1}{N} \sum_{i=1}^N |y_{pred}^i - y_{true}^i| \quad (6.1)$$

```
1  def l1_loss(pred, gt):
2      return torch.abs((pred - gt)).mean()
```

在 Pytorch 中，一般通过 `torch.nn` 实现。

```
1  import torch
2  import torch.nn as nn
3
4  # 默认是 'mean' (平均)，也可设为 'sum' (求和) 或 'none' (保留逐元素误差)
5  criterion = nn.L1Loss(reduction='mean')
6  loss = criterion(y_pred, y_true)
```

2. L2 范数

L2 范数是数学和机器学习中广泛使用的一种“向量长度”或“距离度量”工具，本质上是计算向量元素的平方和的平方根。对于一个向量而言，其 L2 范数表示为公式6.2所示。

$$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2} \quad (6.2)$$

对于论文中的公式6.3，对于向量范数加入了阈值截断，其中 μ 表示高斯球的中心坐标， ϵ 表示设定的阈值。

$$L_{pos} = \max(\|\mu\|_2 - \epsilon_{pos}, 0) \quad (6.3)$$

在代码实现中，对于最值阶段的操作选取了 `F.relu` 激活函数实现，为了约束高四点在局部坐标系的位置不能偏差坐标系原点太大，先计算局部高斯点坐标的范数，然后减去阈值，并通过激活函数判断是否超过阈值，保证在阈值范围内的高斯点是合理的。

```
1 F.relu(gaussians._xyz[visibility_filter].norm(dim=1) - \
2 opt.threshold_xyz).mean()
```

3. L3 范数

L3 范数是 L_p 范数的一种 ($p=3$)，表示向量元素绝对值的三次方之和的三次方根。它在数学上有明确的定义，但在实际应用（如机器学习、优化）中并不常见。对于一个向量而言，其 L3 范数表示为公式6.4所示。

$$\|x\|_3 = (x_1^3 + x_2^3 + \dots + x_n^3)^{1/3} \quad (6.4)$$

4. Scaling Loss

在控制高斯球在 x, y, z 三个方向上的缩放时，应当调整 L2 Norm 的顺序，如公式6.5所示。

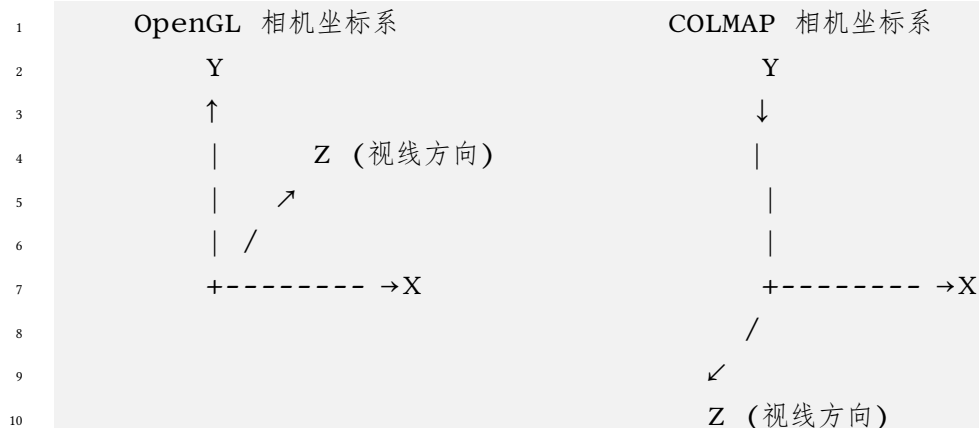
$$L_{scaling} = \|\max(s - \epsilon_{scaling}, 0)\|_2 \quad (6.5)$$

在代码中实现如下，通过指数函数将缩放系数进一步放大，然后对于三个方向的缩放先进行阈值判断，通过激活函数滤除阈值范围内的情况，最后再计算 L2 范数作为损失项。

```
1 F.relu(torch.exp(gaussians._scaling[visibility_filter]) - \
2 opt.threshold_scale).norm(dim=1).mean()
```

6.1.2 Computer Graphics

1. OpenGL/Blender camera axes(Y up, Z back) to COLMAP(Y down, z forward)



```
1 # NeRF 'transform_matrix' is a camera-to-world transform
2 c2w = np.array(frame["transform_matrix"])
3 c2w[:3, 1:3] *= -1
4
5 # get the world-to-camera transform and set R, T
6 w2c = np.linalg.inv(c2w)
7 # R is stored transposed due to 'glm' in CUDA code
8 R = np.transpose(w2c[:3, :3])
9 T = w2c[:3, 3]
```

2. 视场角与焦距的转换

根据前面推导出的公式1.4, 即可推导出相互转换关系。

```
1 def fov2focal(fov, pixels):
2     return pixels / (2 * math.tan(fov / 2))
3
4 def focal2fov(focal, pixels):
5     return 2 * math.atan(pixels / (2 * focal))
```